

aiming-lab / AutoResearchClaw

🔍

+

<> Code

🕒 Issues 2

🔗 Pull requests 1

🤖 Agents

🔄 Actions

📁 Projects

🛡️ Security and quality

📊 Insights

🔔

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

✕

👁 Watch 46

👤

☆

Fully autonomous & self-evolving research from idea to paper. Chat an Idea. Get a Paper. 🦖

📄 MIT license

👤 Contributing

☆ 11.8k stars 🍴 1.4k forks 👁 46 watching 🌿 Branches 🏠 Activity 📋 Custom properties 🏷 Tags

🌐 Public repository

🌿 ... 1 Branch 6 Tags 🔍 Go to file T Go to file Add file ➕ Code ...

👤 Jiaaqiliu

Merge pull request #240 from Maaannnn/feat/volcano

84dad0a · last week

📁 .claude/skills	Add researchclaw skill metadata	last month
📁 docs	Doctor: fix gateway connectivity probe	2 weeks ago
📁 image	Use framework_v2.png to bypass GitHub ...	2 months ago
📁 researchclaw	Merge pull request #240 from Maaannnn...	last week
📁 scripts	fix: deep audit round 4 — 18 bugs across ...	2 months ago
📁 tests	Merge pull request #240 from Maaannnn...	last week
📁 website	fix: security hardening + community feed...	2 months ago
📄 .gitignore	docs: add RESEARCHCLAW_AGENTS.md a...	2 months ago
📄 CONTRIBUTING.md	merge: sync upstream PRs #12, #14, #17,...	2 months ago
📄 LICENSE	docs: add MIT LICENSE file and Discord c...	2 months ago
📄 README.md	Doctor: fix gateway connectivity probe	2 weeks ago
📄 RESEARCHCLAW_AGENTS.md	docs: add RESEARCHCLAW_AGENTS.md a...	2 months ago
📄 RESEARCHCLAW_CLAUDE.md	docs: add RESEARCHCLAW_AGENTS.md a...	2 months ago
📄 config.researchclaw.example.yaml	Doctor: fix gateway connectivity probe	2 weeks ago
📄 config_test_run.yaml	Initial commit	2 months ago
📄 prompts.default.yaml	fix: deep audit rounds 5-6 — 25 bugs fixe...	2 months ago
📄 pyproject.toml	fix: post-merge compatibility fixes for PRs...	2 months ago
📄 sentinel.sh	fix: security hardening + community feed...	2 months ago

📖 README 👤 Contributing 📄 MIT license



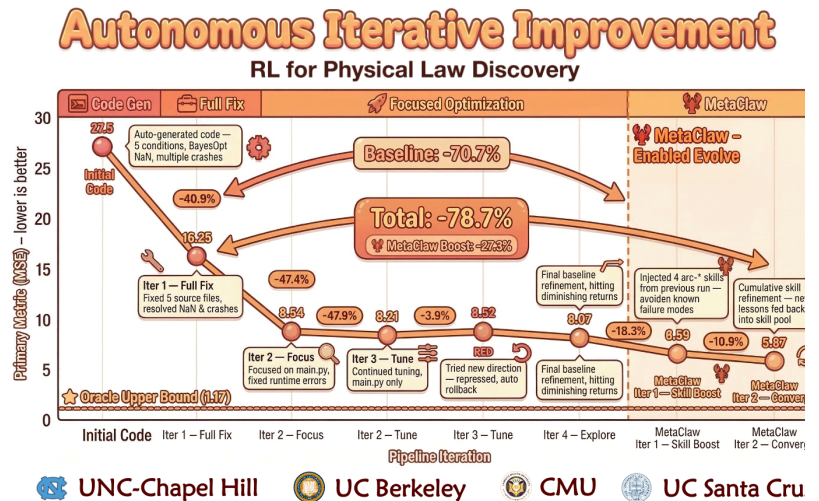
AutoResearchClaw

https://github.com/aiming-lab/AutoResearchClaw

1/13

Chat an Idea. Get a Paper. Autonomous, Collaborative & Self-Evolving.

Just chat with [OpenClaw](#): "Research X" → done.



License [MIT](#) Python [3.11+](#) Tests [2699 passed](#) GitHub [AutoResearchClaw](#) OpenClaw [Compatible](#) Discord [Join Community](#)

[Ethics](#) [Read Before Use](#)

[中文](#) · [日本語](#) · [한국어](#) · [Français](#) · [Deutsch](#) · [Español](#) · [Português](#) · [Русский](#) · [العربية](#)

[Paper Showcase](#) · [Co-Pilot Guide](#) · [Integration Guide](#) · [Discord Community](#)



Generated Paper Showcase

8 papers across 8 domains — math, statistics, biology, computing, NLP, RL, vision, robustness — generated fully autonomously or with Human-in-the-Loop co-pilot guidance.

[VIEW FULL SHOWCASE →](#) [ALL 8 PAPERS](#)

[We're looking for testers!](#) Try the pipeline with your own research idea — from any field — and [tell us what you think](#). Your feedback directly shapes the next version. [→ Testing Guide](#) | [→ 中文测试指南](#) | [→ 日本語テストガイド](#)

News

- **[04/08/2026] Ethics and Responsible Use Guidelines!** — We've added comprehensive [ethics guidelines](#) covering academic integrity, transparency, citation verification, misuse prevention, and dual-use considerations. AI-generated papers are drafts, not finished work — human review is essential. Please read before using AutoResearchClaw for any submission.
- **[04/01/2026] v0.4.0 — Human-in-the-Loop Co-Pilot System** — AutoResearchClaw is no longer purely autonomous. New HITL system adds 6 intervention modes (full-auto , gate-only , checkpoint , step-by-step , co-pilot , custom), per-stage policies, and deep human-AI collaboration. Includes: Idea Workshop for hypothesis co-creation, Baseline Navigator for experiment design review, Paper Co-Writer for collaborative drafting, SmartPause (confidence-driven dynamic intervention), ALHF intervention learning, anti-hallucination claim verification, cost budget guardrails, pipeline branching for parallel hypothesis exploration, and CLI commands (attach / status / approve / reject / guide). [→ Full HITL Guide](#)
- **[03/30/2026] Flexible Skill Loading** — AutoResearchClaw now supports loading open-source and custom skills from any discipline to further enhance your research experience. 19 pre-loaded skills are included as ready-to-use references, covering scientific writing, experiment design, chemistry, biology, and more — including an [A-Evolve](#) agentic evolution skill contributed by the community. Load your own via `researchclaw skills install` or drop a `SKILL.md` into `.claude/skills/`. See [Skills Library](#).
- **[03/22/2026] v0.3.2 — Cross-Platform Support + Major Stability** — AutoResearchClaw now runs on any ACP-compatible agent backend (Claude Code, Codex CLI, Copilot CLI, Gemini CLI, Kimi CLI) and supports messaging platforms (Discord, Telegram, Lark, WeChat) via OpenClaw bridge. New CLI-agent code generation backend delegates Stages 10 & 13 to external CLI agents with budget control and timeout management. Also includes anti-fabrication system (VerifiedRegistry + experiment diagnosis & repair loop), 100+ bug fixes, modular executor refactoring, `--resume` auto-detection, LLM retry hardening, and community-reported fixes.

- ▼ Earlier releases
- [03/18/2026] [v0.3.1](#) — **OpenCode Beast Mode + Community Contributions** — New "Beast Mode" routes complex code generatic to [OpenCode](#) with automatic complexity scoring and graceful fallback. Added Novita AI provider support, thread-safety hardenin improved LLM output parsing robustness, and 20+ bug fixes from community PRs and internal audit.
 - [03/17/2026] [v0.3.0](#) — **MetaClaw Integration** — AutoResearchClaw now supports [MetaClaw](#) cross-run learning: pipeline failures structured lessons → reusable skills, injected into all 23 stages. **+18.3%** robustness in controlled experiments. Opt-in (`metaclaw_bridge.enabled: true`), fully backward-compatible. See [Integration Guide](#).
 - [03/16/2026] [v0.2.0](#) — Three multi-agent subsystems (CodeAgent, BenchmarkAgent, FigureAgent), hardened Docker sandbox wit network-policy-aware execution, 4-round paper quality audit (AI-slop detection, 7-dim review scoring, NeurIPS checklist), and 15+ bug fixes from production runs.
 - [03/15/2026] [v0.1.0](#) — We release AutoResearchClaw: a fully autonomous 23-stage research pipeline that turns a single research idea into a conference-ready paper. No human intervention required.

⚡ One Command. One Paper.

```
# Fully autonomous — no human intervention
pip install -e . && researchclaw setup && researchclaw init && researchclaw run --topic "Your research idea here" --a

# Co-Pilot mode — collaborate with AI at key decision points
researchclaw run --topic "Your research idea here" --mode co-pilot
```

🤔 What Is This?

You think it. AutoResearchClaw writes it. You guide the key decisions.

Drop a research topic — get back a full academic paper with real literature from OpenAlex, Semantic Scholar & arXiv, hardware-aware sandbox experiments (GPU/MPS/CPU auto-detected), statistical analysis, multi-agent peer review, and conference-ready LaTeX targeting NeurIPS/ICML/ICLR. Run it fully autonomous, or use **Co-Pilot mode** to guide the AI at critical decision points — choose research directions, review experiment designs, and co-write the paper. No hallucinated references.

	paper_draft.md	Full academic paper (Introduction, Related Work, Method, Experiments, Results, Conclusion)
	paper.tex	Conference-ready LaTeX (NeurIPS / ICLR / ICML templates)
	references.bib	Real BibTeX references from OpenAlex, Semantic Scholar and arXiv — auto-pruned to match inline citations
	verification_report.json	4-layer citation integrity + relevance verification (arXiv, CrossRef, DataCite, LLM)
	experiment_runs/	Generated code + sandbox results + structured JSON metrics
	charts/	Auto-generated condition comparison charts with error bars and confidence intervals
	reviews.md	Multi-agent peer review with methodology-evidence consistency checks
	evolution/	Self-learning lessons extracted from each run
	deliverables/	All final outputs in one folder — compile-ready for Overleaf

The pipeline runs **end-to-end** — fully autonomous or with human-in-the-loop collaboration. When experiments fail, it self-heals. Whe hypotheses don't hold, it pivots. When citations are fake, it kills them. When you want to steer, it pauses and listens.

🌐 **Run it anywhere.** AutoResearchClaw isn't locked to a single platform. Use it standalone via CLI, plug it into [OpenClaw](#), or wire it up through any ACP-compatible agent —  Claude Code,  Codex CLI,  Copilot CLI,  Gemini CLI,  Kimi CLI, you name it. And because OpenClaw bridges to messaging platforms, you can kick off a full research run from  Discord,  Telegram,  Lark (飞书)  WeChat, or wherever your team already hangs out. One topic in, one paper out — no matter where you type it.

🚀 Quick Start

```
# 1. Clone & install
git clone https://github.com/aiming-lab/AutoResearchClaw.git
cd AutoResearchClaw
python3 -m venv .venv && source .venv/bin/activate
```

```
pip install -e .

# 2. Setup (interactive — installs OpenCode beast mode, checks Docker/LaTeX)
researchclaw setup

# 3. Configure
researchclaw init          # Interactive: choose LLM provider, creates config.arc.yaml
# Or manually: cp config.researchclaw.example.yaml config.arc.yaml

# 4. Run
export OPENAI_API_KEY="sk-..."
researchclaw run --config config.arc.yaml --topic "Your research idea" --auto-approve
```

Output → artifacts/rc-YYYYMMDD-HHMMSS-<hash>/deliverables/ — compile-ready LaTeX, BibTeX, experiment code, charts.

▼ Minimum required config

```
project:
  name: "my-research"

research:
  topic: "Your research topic here"

llm:
  base_url: "https://api.openai.com/v1"
  api_key_env: "OPENAI_API_KEY"
  primary_model: "gpt-4o"
  fallback_models: ["gpt-4o-mini"]

experiment:
  mode: "sandbox"
  sandbox:
    python_path: ".venv/bin/python"
```



 What Makes It Different

Capability	How It Works
 Co-Pilot Mode	6 intervention modes — from fully autonomous to step-by-step. Guide the AI at critical decisions (hypotheses, baselines, paper writing) or let it run free. SmartPause auto-detects when human input would help.
 PIVOT / REFINES Loop	Stage 15 autonomously decides: PROCEED, REFINES (tweak params), or PIVOT (new direction). Artifacts auto-versioned.
 Multi-Agent Debate	Hypothesis generation, result analysis, and peer review each use structured multi-perspective debate.
 Self-Learning	Lessons extracted per run (decision rationale, runtime warnings, metric anomalies) with 30-day time-decay. Future runs learn from past mistakes.
 Knowledge Base	Every run builds structured KB across 6 categories (decisions, experiments, findings, literature, questions, reviews).
 Sentinel Watchdog	Background quality monitor: NaN/Inf detection, paper-evidence consistency, citation relevance scoring, anti-fabrication guard.
 Claim Verification	Inline fact-checking: extracts claims from AI-generated text and cross-references against collected literature. Flags ungrounded citations and fabricated numbers.
 Branch Exploration	Fork the pipeline to explore multiple research directions simultaneously, compare results side-by-side, and merge the best path forward.

 OpenClaw Integration

AutoResearchClaw is an [OpenClaw-compatible service](#). Install it in OpenClaw and launch autonomous research with a single message — or use it standalone via CLI, Claude Code, or any AI coding assistant.

 Use with OpenClaw (Recommended)

If you already use [OpenClaw](#) as your AI assistant:

- 1

Share the GitHub repo URL with OpenClaw
- 2

OpenClaw auto-reads RESEARCHCLAW_AGENTS.md → understands the pipeline
- 3

Say: "Research [your topic]"
- 4

Done – OpenClaw clones, installs, configures, runs, and returns results

That's it. OpenClaw handles `git clone` , `pip install` , config setup, and pipeline execution automatically. You just chat.

►💡 What happens under the hood

🔌 OpenClaw Bridge (Advanced)

For deeper integration, AutoResearchClaw includes a **bridge adapter system** with 6 optional capabilities:

```
# config.arc.yaml
openclaw_bridge:
  use_cron: true           # 🕒 Scheduled research runs
  use_message: true       # 💬 Progress notifications (Discord/Slack/Telegram)
  use_memory: true        # 🧠 Cross-session knowledge persistence
  use_sessions_spawn: true # 🏗️ Spawn parallel sub-sessions for concurrent stages
  use_web_fetch: true     # 🌐 Live web search during literature review
  use_browser: false      # 🖥️ Browser-based paper collection
```

Each flag activates a typed adapter protocol. When OpenClaw provides these capabilities, the adapters consume them without code changes. See [docs/integration-guide.md](#) for full details.

ACP (Agent Client Protocol)

AutoResearchClaw can use **any ACP-compatible coding agent** as its LLM backend — no API keys required. The agent communicates via [acpx](#), maintaining a single persistent session across all 23 pipeline stages.

Agent	Command	Notes
Claude Code	<code>cclaude</code>	Anthropic
Codex CLI	<code>codex</code>	OpenAI
Copilot CLI	<code>gh</code>	GitHub
Gemini CLI	<code>gemini</code>	Google
OpenCode	<code>opencode</code>	SST
Kimi CLI	<code>kimi</code>	Moonshot

```
# config.yaml – ACP example
llm:
  provider: "acp"
  acp:
    agent: "cclaude" # Any ACP-compatible agent CLI command
    cwd: "."         # Working directory for the agent
# No base_url or api_key needed – the agent handles its own auth.
```

```
# Just run – the agent uses its own credentials
researchclaw run --config config.yaml --topic "Your research idea" --auto-approve
```

🔧 Other Ways to Run

Method	How
Standalone CLI	<code>researchclaw run --topic "." --auto-approve</code> (autonomous) or <code>--mode co-pilot</code> (collaborative)
Python API	<code>from researchclaw.pipeline import Runner; Runner(config).run()</code>
Claude Code	Reads <code>RESEARCHCLAW_CLAUDE.md</code> — just say <i>"Run research on [topic]"</i>
Copilot CLI	<code>researchclaw run --topic "."</code> with <code>llm.acp.agent: "gh"</code>
OpenCode	Reads <code>.claude/skills/</code> — same natural language interface

Method	How
Any AI CLI	Provide RESEARCHCLAW_AGENTS.md as context → agent auto-bootstraps

Pipeline: 23 Stages, 8 Phases

Phase A: Research Scoping	Phase E: Experiment Execution
1. TOPIC_INIT	12. EXPERIMENT_RUN
2. PROBLEM_DECOMPOSE	13. ITERATIVE_REFINE ← self-healing
Phase B: Literature Discovery	Phase F: Analysis & Decision
3. SEARCH_STRATEGY	14. RESULT_ANALYSIS ← multi-agent
4. LITERATURE_COLLECT ← real API	15. RESEARCH_DECISION ← PIVOT/REFINE
5. LITERATURE_SCREEN [gate]	Phase G: Paper Writing
6. KNOWLEDGE_EXTRACT	16. PAPER_OUTLINE
Phase C: Knowledge Synthesis	17. PAPER_DRAFT
7. SYNTHESIS	18. PEER_REVIEW ← evidence check
8. HYPOTHESIS_GEN ← debate	19. PAPER_REVISION
Phase D: Experiment Design	Phase H: Finalization
9. EXPERIMENT_DESIGN [gate]	20. QUALITY_GATE [gate]
10. CODE_GENERATION	21. KNOWLEDGE_ARCHIVE
11. RESOURCE_PLANNING	22. EXPORT_PUBLISH ← LaTeX
	23. CITATION_VERIFY ← relevance check

Gate stages (5, 9, 20) pause for human approval or auto-approve with `--auto-approve` . On rejection, the pipeline rolls back.

Co-Pilot mode (`--mode co-pilot`): Deep human-AI collaboration at Stages 7-8 (Idea Workshop), Stage 9 (Baseline Navigator), and Stages 16-17 (Paper Co-Writer). Other stages auto-execute with SmartPause monitoring.

Decision loops: Stage 15 can trigger REFINE (→ Stage 13) or PIVOT (→ Stage 8), with automatic artifact versioning.

►  What Each Phase Does

🌟 Key Features

Feature	Description
 Multi-Source Literature	Real papers from OpenAlex, Semantic Scholar & arXiv — query expansion, deduplication, circuit breaker with graceful degradation
 4-Layer Citation Verification	arXiv ID check → CrossRef/DataCite DOI → Semantic Scholar title match → LLM relevance scoring. Hallucinated refs auto-removed.
 Hardware-Aware Execution	Auto-detects GPU (NVIDIA CUDA / Apple MPS / CPU-only) and adapts code generation, imports, and experiment scale accordingly
 OpenCode Beast Mode	Complex experiments auto-routed to OpenCode — generates multi-file projects with custom architectures, training loops, and ablation studies. Install via <code>researchclaw setup</code> .
 Sandbox Experiments	AST-validated code, immutable harness, NaN/Inf fast-fail, self-healing repair, iterative refinement (up to 10 rounds), partial result capture
 Conference-Grade Writing	NeurIPS/ICML/ICLR templates, section-by-section drafting (5,000-6,500 words), anti-fabrication guard, revision length guard, anti-disclaimer enforcement
 Template Switching	<code>neurips_2025</code> , <code>iclr_2026</code> , <code>icml_2026</code> — Markdown → LaTeX with math, tables, figures, cross-refs, <code>\cite{}</code>
 Anti-Fabrication	VerifiedRegistry enforces ground-truth experiment data in papers. Auto-diagnoses failed experiments and repairs them before writing. Unverified numbers sanitized.
 Quality Gates	3 human-in-the-loop gates (Stages 5, 9, 20) with rollback. Skip with <code>--auto-approve</code> .
 HITL Co-Pilot	6 intervention modes with per-stage policies. Idea Workshop, Baseline Navigator, Paper Co-Writer for deep collaboration. SmartPause, cost guardrails, escalation policies, and intervention learning for production safety. CLI/WebSocket/MCP adapters.

Feature	Description
💰 Cost Guardrails	Budget monitoring with configurable threshold alerts (50%/80%/100%). Pipeline auto-pauses when cost exceeds budget.
🔒 Reproducibility	SHA256 checksums for all stage artifacts. Immutable manifests for verification. Multi-level undo with versioned snapshots.

Human-in-the-Loop Co-Pilot

AutoResearchClaw v0.4.0 introduces a complete Human-in-the-Loop (HITL) system that transforms the pipeline from purely autonomous to a human-AI collaborative research engine. Choose your level of involvement:

Intervention Modes

Mode	Command	What It Does
Full Auto	--auto-approve	Original behavior — no human intervention
Gate Only	--mode gate-only	Pause at 3 gate stages (5, 9, 20) for approval
Checkpoint	--mode checkpoint	Pause at each phase boundary (8 checkpoints)
Co-Pilot	--mode co-pilot	Deep collaboration at critical stages, auto elsewhere
Step-by-Step	--mode step-by-step	Pause after every stage — learn the pipeline
Express	--mode express	Quick review — only 3 most critical gates
Custom	--mode custom	Define per-stage policies via stage_policies config

Co-Pilot Workflow

You: `researchclaw run --topic "Quantum noise as neural network regularization" --mode co-pilot`

Pipeline runs Stages 1-7 automatically...

```
HITL | Stage 08: HYPOTHESIS_GEN
Post-stage review

Hypotheses mentioned: 3
Novelty score: 0.72 (moderate)

[a] Approve [r] Reject [e] Edit [c] Collaborate
[i] Inject guidance [v] View output [q] Abort
```

You: `c` (start collaborative chat)
You: Hypothesis 3 is interesting but needs Dropout/Label Smoothing as baselines
AI: Updated — added Dropout, Label Smoothing, MixUp, CutMix as baselines...
You: `approve`
Pipeline continues with your refined hypothesis...

CLI Commands

```
# Start with HITL mode
researchclaw run --topic "..." --mode co-pilot

# Attach to a paused pipeline (from another terminal)
researchclaw attach artifacts/rc-2026-xxx

# Check pipeline and HITL status
researchclaw status artifacts/rc-2026-xxx

# Approve/reject from another terminal or script
researchclaw approve artifacts/rc-2026-xxx --message "LGTM"
researchclaw reject artifacts/rc-2026-xxx --reason "Missing key baseline"
```

```
# Inject guidance for a stage (even before it runs)
researchclaw guide artifacts/rc-2026-xxx --stage 9 --message "Use ResNet-50 as primary baseline"
```

Key Capabilities

Feature	Description
Idea Workshop	Brainstorm, evaluate, and refine hypotheses collaboratively (Stage 7-8)
Baseline Navigator	AI suggests baselines + human adds/removes + reproducibility checklist (Stage 9)
Paper Co-Writer	Section-by-section drafting with human editing and AI polishing (Stage 16-19)
SmartPause	Confidence-driven dynamic pausing — auto-detects when human input would help
Claim Verification	Inline fact-checking against collected literature — flags ungrounded claims
Cost Guardrails	Budget monitoring with 50%/80%/100% threshold alerts
Intervention Learning	ALHF — learns from your review patterns to optimize future pause decisions
Branch Exploration	Fork pipeline to explore multiple hypotheses, compare, merge the best
Escalation Policy	Tiered notification (terminal → Slack → email → auto-halt) when unattended
3 Adapters	CLI (terminal), WebSocket (web dashboard), MCP (external agents)

Configuration

```
# config.arc.yaml
hitl:
  enabled: true
  mode: co-pilot # full-auto | gate-only | checkpoint | co-pilot | custom
  cost_budget_usd: 50.0 # Pause when cost exceeds budget (0 = no limit)

notifications:
  on_pause: true
  on_quality_drop: true
  channels: ["terminal"] # terminal | slack | webhook

timeouts:
  default_human_timeout_sec: 86400 # 24h default wait
  auto_proceed_on_timeout: false

collaboration:
  max_chat_turns: 50
  save_chat_history: true

# Per-stage custom policies (optional, for 'custom' mode)
stage_policies:
  8: { require_approval: true, enable_collaboration: true }
  9: { require_approval: true, allow_edit_output: true }
```

Backward Compatibility

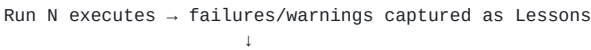
- **Default: OFF.** Without `hitl.enabled: true` or `--mode`, the pipeline behaves exactly as before.
- `--auto-approve` **still works.** It overrides HITL mode.
- **All 2,699 existing tests pass** with HITL code present.

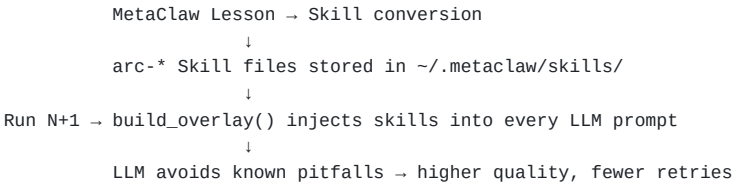
🧠 MetaClaw Integration

AutoResearchClaw + [MetaClaw](#) = A pipeline that learns from every run.

MetaClaw adds **cross-run knowledge transfer** to AutoResearchClaw. When enabled, the pipeline automatically captures lessons from failures and warnings, converts them into reusable skills, and injects those skills into all 23 pipeline stages on subsequent runs — so the same mistakes are never repeated.

How It Works





Quick Setup

```
# 1. Install MetaClaw (if not already)
pip install metaclaw

# 2. Enable in your config

# config.arc.yaml
metaclaw_bridge:
  enabled: true
  proxy_url: "http://localhost:30000"           # MetaClaw proxy (optional)
  skills_dir: "~/.metaclaw/skills"             # Where skills are stored
  fallback_url: "https://api.openai.com/v1"    # Direct LLM fallback
  fallback_api_key: ""                        # API key for fallback URL
  lesson_to_skill:
    enabled: true
    min_severity: "warning"                   # Convert warnings + errors
    max_skills_per_run: 3

# 3. Run as usual – MetaClaw works transparently
researchclaw run --config config.arc.yaml --topic "Your idea" --auto-approve
```

After each run, check `~/.metaclaw/skills/arc-*/SKILL.md` to see the skills your pipeline has learned.

Experiment Results

In controlled A/B experiments (same topic, same LLM, same configuration):

Metric	Baseline	With MetaClaw	Improvement
Stage retry rate	10.5%	7.9%	-24.8%
Refine cycle count	2.0	1.2	-40.0%
Pipeline stage completion	18/19	19/19	+5.3%
Overall robustness score (composite)	0.714	0.845	+18.3%

Composite robustness score is a weighted average of stage completion rate (40%), retry reduction (30%), and refine cycle efficiency (30%).

Backward Compatibility

- **Default: OFF.** If `metaclaw_bridge` is absent or `enabled: false`, the pipeline behaves exactly as before.
- **No new dependencies.** MetaClaw is optional — the core pipeline works without it.
- **All 2,699 existing tests pass** with the integration code present.

 Skills Library

AutoResearchClaw now supports loading **open-source and custom skills** to further enhance your research experience. We also ship with **19 pre-loaded built-in skills** (scientific writing, literature search, chemistry, biology, and more) as ready-to-use references, offering a high degree of flexibility out of the box. Disable any skill by adding `enabled: false` to its frontmatter.

Sample built-in skills:

Category	Skill	Description
Writing	scientific-writing	IMRAD structure, citation formatting, reporting guidelines

Category	Skill	Description
Domain	chemistry-rdkit	Molecular analysis, SMILES, fingerprints, drug discovery
Experiment	literature-search	Systematic review, PRISMA methodology

See all 19 skills with `researchclaw skills list`.

Load Your Own Skills

```
# Option 1: Install a skill (persists across projects)
researchclaw skills install /path/to/my-skill/

# Option 2: Drop a SKILL.md into the project
mkdir -p .claude/skills/my-custom-skill
# Then create a SKILL.md with YAML frontmatter (name, description, trigger-keywords, applicable-stages)

# Option 3: Configure shared skill directories in config.arc.yaml
# skills:
#   custom_dirs:
#     - /path/to/team-shared-skills
```

Using Skills

Skills are loaded and injected into LLM prompts automatically — no manual activation needed. Use the CLI to inspect:

```
researchclaw skills list           # Show all loaded skills with sources
researchclaw skills validate ./my-skill # Check SKILL.md format
```

Browse community skills: [K-Dense-AI/claude-scientific-skills](#) (150+ scientific skills across multiple disciplines).

⚙️ Configuration Reference

▼ Click to expand full configuration reference

```
# === Project ===
project:
  name: "my-research"           # Project identifier
  mode: "docs-first"           # docs-first | semi-auto | full-auto

# === Research ===
research:
  topic: "..."                # Research topic (required)
  domains: ["ml", "nlp"]        # Research domains for literature search
  daily_paper_count: 8          # Target papers per search query
  quality_threshold: 4.0        # Minimum quality score for papers

# === Runtime ===
runtime:
  timezone: "America/New_York"  # For timestamps
  max_parallel_tasks: 3         # Concurrent experiment limit
  approval_timeout_hours: 12    # Gate stage timeout
  retry_limit: 2                # Retry count on stage failure

# === LLM ===
llm:
  provider: "openai-compatible" # openai | openrouter | deepseek | minimax | volcengine | volcengine-coding-plan |
  base_url: "https://..."     # API endpoint (required for openai-compatible)
  api_key_env: "OPENAI_API_KEY" # Env var for API key (required for openai-compatible)
  api_key: ""                   # Or hardcoded key here
  primary_model: "gpt-4o"        # Primary model
  fallback_models: ["gpt-4o-mini"] # Fallback chain
  s2_api_key: ""                # Semantic Scholar API key (optional, higher rate limits)
  acp:                          # Only used when provider: "acp"
    agent: "claude"             # ACP agent CLI command (claude, codex, gemini, etc.)
    cwd: "."                   # Working directory for the agent

# Volcengine / BytePlus presets via `researchclaw init`
#   volcengine                 -> VOLCENGINE_API_KEY
#   volcengine-coding-plan     -> VOLCENGINE_API_KEY
#   byteplus                   -> BYTEPLUS_API_KEY
#   byteplus-coding-plan       -> BYTEPLUS_API_KEY
```

```

# === Experiment ===
experiment:
  mode: "sandbox" # simulated | sandbox | docker | ssh_remote
  time_budget_sec: 300 # Max execution time per run (default: 300s)
  max_iterations: 10 # Max optimization iterations
  metric_key: "val_loss" # Primary metric name
  metric_direction: "minimize" # minimize | maximize
  sandbox:
    python_path: ".venv/bin/python"
    gpu_required: false
    allowed_imports: [math, random, json, csv, numpy, torch, sklearn]
    max_memory_mb: 4096
  docker:
    image: "researchclaw/experiment:latest"
    network_policy: "setup_only" # none | setup_only | pip_only | full
    gpu_enabled: true
    memory_limit_mb: 8192
    auto_install_deps: true # Auto-detect imports → requirements.txt
  ssh_remote:
    host: "" # GPU server hostname
    gpu_ids: [] # Available GPU IDs
    remote_workdir: "/tmp/researchclaw_experiments"
  opencode: # OpenCode Beast Mode (auto-installed via `researchclaw setup`)
    enabled: true # Master switch (default: true)
    auto: true # Auto-trigger without confirmation (default: true)
    complexity_threshold: 0.2 # 0.0-1.0 – higher = only trigger on complex experiments
    model: "" # Override model (empty = use llm.primary_model)
    timeout_sec: 600 # Max seconds for OpenCode generation
    max_retries: 1 # Retry count on failure
    workspace_cleanup: true # Remove temp workspace after collection
  code_agent: # CodeAgent v2 – multi-phase code generation
    enabled: true # Use CodeAgent instead of legacy single-prompt codegen
    architecture_planning: true # Generate deep implementation blueprint before coding
    sequential_generation: true # Generate files one-by-one following dependency DAG
    hard_validation: true # AST-based validation gates (blocks identical ablations, hardcoded metrics)
    hard_validation_max_repairs: 2 # Max repair attempts when validation fails
    exec_fix_max_iterations: 3 # Execution-in-the-loop fix attempts
    exec_fix_timeout_sec: 60 # Timeout per exec-fix attempt
  benchmark_agent: # BenchmarkAgent – automated dataset & baseline selection
    enabled: true # Enable 4-agent benchmark pipeline (Surveyor–Selector–Acquirer–Validator)
    enable_hf_search: true # Search HuggingFace Datasets
    enable_web_search: true # Search Google Scholar for benchmarks
    tier_limit: 2 # Dataset tier filtering (1=small/cached, 2=medium, 3=large)
    min_benchmarks: 1 # Minimum datasets required
    min_baselines: 2 # Minimum baseline methods required
  figure_agent: # FigureAgent – academic figure generation
    enabled: true # Enable 5-agent figure pipeline (Planner–CodeGen–Renderer–Critic–Integrator)
    min_figures: 3 # Minimum figures to generate
    max_figures: 8 # Maximum figures
    max_iterations: 3 # Critic-driven refinement iterations
    dpi: 300 # Output resolution
    strict_mode: false # Fail pipeline if figure generation fails
  repair: # Anti-fabrication experiment repair
    enabled: true # Auto-diagnose and repair failed experiments
    max_cycles: 3 # Repair retry loops
    min_completion_rate: 0.5 # >=50% conditions must complete to proceed
    min_conditions: 2 # At least 2 conditions for valid experiment
    use_opencode: true # Route repairs through OpenCode Beast Mode

# === Web Search (Optional) ===
web_search:
  enabled: true # Enable web-augmented literature search
  tavily_api_key_env: "TAVILY_API_KEY" # Tavily API key env var (optional)
  enable_scholar: true # Google Scholar search
  enable_pdf_extraction: true # Extract text from PDFs
  max_web_results: 10 # Max web results per query

# === Export ===
export:
  target_conference: "neurips_2025" # neurips_2025 | iclr_2026 | icml_2026
  authors: "Anonymous"
  bib_file: "references"

# === Prompts ===
prompts:
  custom_file: "" # Path to custom prompts YAML (empty = defaults)

# === HITL Co-Pilot (NEW in v0.4.0) ===

```

```
hitl:
  enabled: false                # Set to true to enable HITL
  mode: co-pilot                # full-auto | gate-only | checkpoint | step-by-step | co-pilot | custom
  cost_budget_usd: 0.0          # Cost limit in USD (0 = no limit)
  notifications:
    on_pause: true              # Notify when pipeline pauses
    on_quality_drop: true       # Notify on quality issues
    channels: ["terminal"]      # terminal | slack | webhook
  timeouts:
    default_human_timeout_sec: 86400 # Wait up to 24h for human input
    auto_proceed_on_timeout: false   # If true, auto-approve on timeout
  collaboration:
    max_chat_turns: 50          # Max turns per collaboration session
    save_chat_history: true     # Persist chat logs
    stage_policies: {}          # Per-stage overrides (for 'custom' mode)

# === Security ===
security:
  hitl_required_stages: [5, 9, 20] # Stages requiring human approval
  allow_publish_without_approval: false
  redact_sensitive_logs: true

# === Knowledge Base ===
knowledge_base:
  backend: "markdown"          # markdown | obsidian
  root: "docs/kb"

# === Notifications ===
notifications:
  channel: "console"           # console | discord | slack
  target: ""

# === MetaClaw Bridge (Optional) ===
metaclaw_bridge:
  enabled: false               # Set to true to enable cross-run learning
  proxy_url: "http://localhost:30000" # MetaClaw proxy URL
  skills_dir: "~/metaclaw/skills" # Where arc-* skills are stored
  fallback_url: ""             # Direct LLM fallback when proxy is down
  fallback_api_key: ""         # API key for fallback endpoint
  lesson_to_skill:
    enabled: true              # Auto-convert lessons to skills
    min_severity: "warning"    # Minimum severity to convert
    max_skills_per_run: 3      # Max new skills per pipeline run
  prm:
    enabled: false             # Process Reward Model quality gate (optional)
    model: "gpt-5.4"           # Use LLM-as-judge to score stage outputs
    votes: 3                   # PRM judge model
    gate_stages: [5, 9, 15, 20] # Majority vote count
    # Stages to apply PRM gates

# === OpenClaw Bridge ===
openclaw_bridge:
  use_cron: false              # Scheduled research runs
  use_message: false           # Progress notifications
  use_memory: false            # Cross-session knowledge persistence
  use_sessions_spawn: false    # Spawn parallel sub-sessions
  use_web_fetch: false         # Live web search
  use_browser: false           # Browser-based paper collection
```

Acknowledgments

Inspired by:

-  [AI Scientist](#) (Sakana AI) — Automated research pioneer
-  [AutoResearch](#) (Andrej Karpathy) — End-to-end research automation
-  [FARS](#) (Analemma) — Fully Automated Research System

Ethics and Responsible Use

AutoResearchClaw is a research assistance tool, not a replacement for human researchers. We ask all users to observe the following principles:

Academic integrity. Papers generated by AutoResearchClaw should be treated as drafts that require thorough human review, verification, and revision before any submission. Authors listed on a paper bear full responsibility for its content, claims, and correctness. Using AI-generated text without adequate human oversight or disclosure may violate academic integrity policies at your institution or target venue.

Transparency and disclosure. We strongly encourage users to disclose the use of AutoResearchClaw (or any AI assistance) in their manuscripts, in accordance with the policies of the target venue (e.g., NeurIPS, ICML, ICLR, and most major venues now require disclosure of AI writing assistance). The Human-in-the-Loop Co-Pilot exists precisely to keep humans in meaningful control of research decisions.

Citation and attribution. AutoResearchClaw verifies citations through a 4-layer pipeline, but no automated system is perfect. Users must manually verify that all references are real, relevant, and correctly cited before submission. Fabricated or misattributed citations undermine scientific trust.

Potential for misuse. Like any powerful tool, AutoResearchClaw can be misused to produce low-quality or misleading research at scale. We do not condone using this system to generate paper mills, fraudulent submissions, or content designed to game peer review. We reserve the right to update the license or terms of use if systematic misuse is identified.

Dual use. Autonomous research systems raise broader questions about the future of scientific labor, authorship norms, and review processes. We welcome community discussion on these topics and are committed to developing this technology responsibly.

By using AutoResearchClaw, you agree to use it in a manner consistent with these principles and with the ethical guidelines of your institution and research community.

License

MIT — see [LICENSE](#) for details.

Releases 6

 **v0.4.0: Human-in-the-Loop Co-Pilot System** Latest
last month

[+ 5 releases](#)

Packages

No packages published

Contributors 35



[+ 21 contributors](#)

Languages

● Python 97.0% ● HTML 1.5% ● TeX 0.5% ● BibTeX Style 0.4% ● CSS 0.3% ● Shell 0.2% ● Dockerfile 0.1%